Unified, Branch-Free, and C^∞-Smooth Stability Formulations for Boundary Layer Physics

A Technical Note on Regularization, Derivative Discontinuities, and SIMD Vectorization

Atmospheric Physics & Computational Modeling Group
University of Alabama in Huntsville (UAH)

August 18, 2026

Abstract

Atmospheric boundary layer (ABL) schemes in weather and climate models frequently rely on Richardson number (Ri) stability functions $S_m(\mathrm{Ri})$ and $S_h(\mathrm{Ri})$ defined piecewise across unstable ($\mathrm{Ri} < 0$), neutral ($\mathrm{Ri} = 0$), and stable ($0 < \mathrm{Ri} < \mathrm{Ri}_c$) regimes. Conditional branching in these formulations introduces instruction divergence on modern SIMD/GPU architectures and derivative kinks that can destabilize high-order numerical solvers. Here, we analyze algebraic splitting techniques to unify these regimes into a single, branch-free expression. We demonstrate why naive hyperbolic tangent blending inherits derivative kinks if hard clipping is retained inside branch functions, and present two mathematically rigorous C^∞-smooth remedies: a zero-offset hyperbolic regularization that preserves double-precision accuracy at neutral stability without numerical drift, and a clean NaN-guarded sigmoid blend.

1 Introduction and Physical Context

In Monin–Obukhov Similarity Theory (MOST) and higher-order boundary layer turbulence closures (e.g., Mellor–Yamada, EDMF, and WRF boundary layer schemes), stability functions $S_m(\mathrm{Ri})$ and $S_h(\mathrm{Ri})$ modify turbulent eddy diffusivities for momentum and heat based on the gradient Richardson number:

$$\mathrm{Ri} = \frac{\frac{g}{\theta_0}\frac{\partial \theta_v}{\partial z}}{\left(\frac{\partial u}{\partial z}\right)^2 + \left(\frac{\partial v}{\partial z}\right)^2}. \tag{1}$$

Traditionally, models handle regime transitions using explicit `if-else` control logic:

$$S_m(\mathrm{Ri}) = \begin{cases} (1 - B_u \mathrm{Ri})^{1/2}, & \mathrm{Ri} < 0 \quad \text{(Unstable)}, \\ 1.0, & \mathrm{Ri} = 0 \quad \text{(Neutral)}, \\ \left(1 - \frac{\mathrm{Ri}}{\mathrm{Ri}_c}\right)^2, & 0 < \mathrm{Ri} < \mathrm{Ri}_c \quad \text{(Stable)}, \\ 0, & \mathrm{Ri} \geq \mathrm{Ri}_c \quad \text{(Critically Stable)}. \end{cases} \tag{2}$$

While computationally simple on legacy single-core CPU architectures, explicit branching introduces severe performance penalties on massively parallel GPU architectures (e.g., NVIDIA CUDA, AMD ROCm) and vector execution units (AVX-512) due to thread divergence. Furthermore, derivative discontinuities (C^0 kinks) at regime boundaries can impair the convergence of implicit time-stepping schemes, Newton–Raphson solvers, and adjoint-based data assimilation pipelines.

2 The Absolute-Value Master Formulation

To eliminate conditional branching, we define algebraic component splitters using the absolute value function:

$$\mathrm{Ri}_+ = \frac{\mathrm{Ri} + |\mathrm{Ri}|}{2} = \max(\mathrm{Ri}, 0), \qquad \mathrm{Ri}_- = \frac{\mathrm{Ri} - |\mathrm{Ri}|}{2} = \min(\mathrm{Ri}, 0). \tag{3}$$

Substituting Ri_+ and Ri_- into a unified master expression yields:

$$S_m(\mathrm{Ri}) = \left[1 - \frac{\mathrm{Ri} + |\mathrm{Ri}|}{2\mathrm{Ri}_c}\right]^2 \left[1 - B_u \left(\frac{\mathrm{Ri} - |\mathrm{Ri}|}{2}\right)\right]^{1/2}, \tag{4}$$

$$S_h(\mathrm{Ri}) = \frac{1}{\alpha_\theta}\left[1 - \frac{\mathrm{Ri} + |\mathrm{Ri}|}{2\mathrm{Ri}_c}\right]^2 \left[1 - B_u \left(\frac{\mathrm{Ri} - |\mathrm{Ri}|}{2}\right)\right]^{3/4}. \tag{5}$$

2.1 Regime Verification

Evaluating Eqs. (4)–(5) across stability regimes confirms exact algebraic equivalence to the piecewise definition:

- **Unstable ($\mathbf{Ri} < 0$):** $|\mathrm{Ri}| = -\mathrm{Ri} \implies \mathrm{Ri}_+ = 0$, $\mathrm{Ri}_- = \mathrm{Ri}$. The stable factor evaluates to 1.0, leaving $S_m = (1 - B_u\mathrm{Ri})^{1/2}$.
- **Neutral ($\mathbf{Ri} = 0$):** $\mathrm{Ri}_+ = \mathrm{Ri}_- = 0$. Both factors evaluate to 1.0, recovering $S_m(0) = 1.0$ and $S_h(0) = 1/\alpha_\theta$.
- **Stable ($0 < \mathbf{Ri} < \mathbf{Ri}_c$):** $|\mathrm{Ri}| = \mathrm{Ri} \implies \mathrm{Ri}_+ = \mathrm{Ri}, \mathrm{Ri}_- = 0$. The unstable factor evaluates to 1.0, leaving the quadratic cutoff $(1 - \mathrm{Ri}/\mathrm{Ri}_c)^2$.

2.2 Differentiability Analysis (C^0 vs. C^1)

Although continuous (C^0), the derivative of $|\mathrm{Ri}|$ at $\mathrm{Ri} = 0$ possesses a jump discontinuity. Examining one-sided limits at neutral stability:

$$\left.\frac{dS_m}{d\mathrm{Ri}}\right|_{\mathrm{Ri}\to 0^+} = -\frac{2}{\mathrm{Ri}_c}, \qquad \left.\frac{dS_m}{d\mathrm{Ri}}\right|_{\mathrm{Ri}\to 0^-} = -\frac{B_u}{2}, \tag{6}$$

$$\left.\frac{dS_h}{d\mathrm{Ri}}\right|_{\mathrm{Ri}\to 0^+} = -\frac{2}{\alpha_\theta \mathrm{Ri}_c}, \qquad \left.\frac{dS_h}{d\mathrm{Ri}}\right|_{\mathrm{Ri}\to 0^-} = -\frac{3B_u}{4\alpha_\theta}. \tag{7}$$

Thus, achieving C^1 smoothness across $\mathrm{Ri} = 0$ requires variable-specific coupling constants:

$$B_{u,m} = \frac{4}{\mathrm{Ri}_c}, \qquad B_{u,h} = \frac{8}{3\mathrm{Ri}_c}. \tag{8}$$

If B_u is treated as a global constant, derivative kinks persist at neutral stability. Moreover, higher-order derivatives (C^2) remain discontinuous.

3 Pitfall Analysis: Structural Flaw in Naive Sigmoid Blending

A common attempt to achieve C^∞ smoothness involves blending branch expressions using a hyperbolic tangent sigmoid weight $w(\mathrm{Ri}) = \frac{1}{2}\left[1 + \tanh\left(\frac{\mathrm{Ri}}{\delta}\right)\right]$. However, a critical design error occurs when hard clipping (Ri_+) is retained inside the stable component:

$$S_m^{\text{naive}}(\mathrm{Ri}) = [1 - w(\mathrm{Ri})] \cdot (1 - B_u\mathrm{Ri})^{1/2} + w(\mathrm{Ri}) \cdot \left(1 - \frac{\mathrm{Ri}_+}{\mathrm{Ri}_c}\right)^2. \tag{9}$$

Theorem: The Tanh Derivative Leak

The naive formulation in Eq. (9) **fails to achieve C^1 continuity**, regardless of the transition width parameter δ.

Proof: Let $f_{\text{stable}}(\text{Ri}) = \left(1 - \frac{\text{Ri}_+}{\text{Ri}_c}\right)^2$. Over all Ri, f_{stable} contains a hard clip at $\text{Ri} = 0$. Its one-sided derivatives are:

$$f'_{\text{stable}}(0^-) = 0, \qquad f'_{\text{stable}}(0^+) = -\frac{2}{\text{Ri}_c}. \tag{10}$$

Applying the product rule to $S_m^{\text{naive}}(\text{Ri})$ at $\text{Ri} = 0$, noting $w(0) = \frac{1}{2}$ and $w'(0) = \frac{1}{2\delta}$:

$$S'_m(0^-) = [1 - w(0)]\left(-\frac{B_u}{2}\right) + w'(0)(1) - w'(0)(1) + w(0)(0) = -\frac{B_u}{4}, \tag{11}$$

$$S'_m(0^+) = -\frac{B_u}{4} + w(0) f'_{\text{stable}}(0^+) = -\frac{B_u}{4} - \frac{1}{\text{Ri}_c}. \tag{12}$$

The slope jump across neutral stability is:

$$\Delta S'_m = S'_m(0^+) - S'_m(0^-) = -\frac{1}{\text{Ri}_c} \neq 0. \tag{13}$$

Because $w(0) = 0.5$, the hard clip inside f_{stable} leaks a derivative discontinuity of exactly $1/\text{Ri}_c$ into the composite function. Shrinking or widening δ cannot fix this structural flaw.

4 Mathematically Sound C^∞ Solutions

4.1 Solution 1: Zero-Offset Hyperbolic Regularization

Replacing $|\text{Ri}|$ with the smooth hyperbola $\sqrt{\text{Ri}^2 + \epsilon^2}$ renders all components real-analytic (C^∞). However, a naive substitution introduces an $O(\epsilon)$ deviation at neutral stability ($\text{Ri} = 0 \implies \text{Ri}_+ \approx \epsilon/2$), which degrades precision unless $\epsilon < 10^{-8}$.

To guarantee **exact neutral stability recovery** to double-precision tolerance for any ϵ, we introduce the **Zero-Offset Shift**:

Zero-Offset Hyperbolic Regularizer

$$\text{Ri}_+ = \frac{\text{Ri} + \sqrt{\text{Ri}^2 + \epsilon^2} - \epsilon}{2}, \tag{14}$$

$$\text{Ri}_- = \frac{\text{Ri} - \sqrt{\text{Ri}^2 + \epsilon^2} + \epsilon}{2}. \tag{15}$$

4.1.1 Key Properties

1. **Exact Neutral Point:** At $\text{Ri} = 0$, $\text{Ri}_+ = \frac{0+\epsilon-\epsilon}{2} = 0$ and $\text{Ri}_- = 0$, yielding $S_m(0) = 1.0$ and $S_h(0) = 1/\alpha_\theta$ with zero error.

2. **Smooth Derivatives:** The neutral slope evaluates continuously to:

$$\left.\frac{dS_m}{d\text{Ri}}\right|_{\text{Ri}=0} = -\frac{1}{\text{Ri}_c} - \frac{B_u}{4}. \tag{16}$$

3. **Numerical Conditioning:** Selecting $\epsilon \approx 10^{-3}$ to 10^{-4} provides optimal C^∞ smoothing without stiffening differential equation solvers.

4.2 Solution 2: Clean Sigmoid Blending with IEEE NaN Guard

To correct the sigmoid blend, we remove the clip Ri_+ completely, allowing both branch functions to remain smooth unclipped polynomials near $\mathrm{Ri} = 0$. To prevent floating-point `NaN` propagation when evaluating $(1 - B_u\mathrm{Ri})^{1/2}$ for large positive Ri, we introduce a small floor parameter $\eta \approx 10^{-12}$:

Clean IEEE-Guarded Sigmoid Blend

$$S_m(\mathrm{Ri}) = [1 - w(\mathrm{Ri})]\sqrt{\max\left(\eta,\, 1 - B_u\mathrm{Ri}\right)} + w(\mathrm{Ri})\left(1 - \frac{\mathrm{Ri}}{\mathrm{Ri}_c}\right)^2, \tag{17}$$

where $w(\mathrm{Ri}) = \frac{1}{2}\left[1 + \tanh\left(\frac{\mathrm{Ri}}{\delta}\right)\right]$.

Since both branch functions are smooth everywhere in the neighborhood of $\mathrm{Ri} = 0$, and $w(\mathrm{Ri}) \in C^\infty$, the resulting combination is genuinely C^∞.

5 Comparative Evaluation & Recommendations

Table 1: Comparison of Unified Stability Function Formulations

Formulation	**Continuity**	$S_m(0)$ **Accuracy**	**SIMD/GPU**	**Primary Advantage / Trac**
Piecewise Absolute Split	C^0	Exact (1.0)	Excellent	Simple, but derivative kink
Naive Tanh Blend	C^0	Exact (1.0)	Good	Broken: Leaks $1/\mathrm{Ri}_c$ slope l
Zero-Offset Hyperbolic	C^∞	Exact (1.0)	Optimal	Smooth, exact at 0, 1 `sqrt` F
Clean Sigmoid Blend	C^∞	Exact (1.0)	High	Smooth, needs IEEE `NaN` fl

6 Conclusion

For modern high-performance GPU atmospheric models (e.g., CUDA-accelerated boundary layer schemes), the **Zero-Offset Hyperbolic Regularization** (Eqs. (14)–(15)) offers the optimal blend of exact physical accuracy at neutral stability, complete C^∞ differentiability, branch-free vector execution, and computational efficiency.